THE LUXE MVP - Technical Summary

Executive Summary

We'll build **THE LUXE MVP** as a **Django (API + domain logic) + Next.js (creator cockpit UI)** app with a **Model Router** that supports **local LLM (RTX 4060 Ti)** first and can switch to hosted models later. MVP focuses on **IG + TikTok structured outputs** (caption/script/shot list/etc.), **compliance traffic-light**, **audit/versioning**, and a **progress dashboard** that tracks content from *brief → draft → check → revise → ready*.

Extracted Requirements

Functional

- **Workspaces + Users + Roles** (solo creator MVP; keep RBAC minimal but present)
- **Brand Voice Profile v1**
    - Inputs: pasted examples, uploaded files (later), do/don't lists
    - Outputs: tone rules, vocabulary, banned phrases/claims, disclaimers, "brand facts"
- **Brief → Generation**
    - Instagram: caption variants + hook + CTA + hashtags
    - TikTok: script + shot list + on-screen text + caption + hashtags
- **Compliance Traffic Light v1**
    - Rules for: consent/privacy, medical/financial claims phrasing, IP/trademark red flags, disclosure reminders:
        - Location based, a way to extract per country or region conditions
    - Explainable findings + "confirm/override with reason"
- **Progress Dashboard**
    - Pipeline status per post/project
    - Basic metrics: drafts created, revisions, time-in-stage
- **Audit Trail**
    - Version history for drafts
    - Who changed what + compliance decisions
- Try to test API using burner accounts

Non-functional

- Privacy by design (minimal sensitive storage), logs without leaking content by default

- Find a way to do analysis without too much memorization, how to keep brand knowledge local
- Stateless analysis
- Stateful Brand Knowledge
- Cost control: caching + quotas + model routing
- Maintainability: modular Django apps + typed schemas
- “Local-first inference” support and easy swap to hosted models

---

MVP Definition

MVP Must-haves

- Creator workspace (single workspace per user for now)
- BrandProfile v1 (CRUD + optional “extract” action)
- Brief creation
- Draft generation (IG + TikTok)
- Compliance check + confirm/override
- Progress board (kanban-ish stages)
- Export copy-ready blocks

V1 (after MVP)

- Multi-brand per creator + templates library
- RAG “facts-first” grounding using brand facts
- Collaborators + approvals
- Scheduling integrations

V2 (scale)

- Agency multi-client workspaces
- Multi-model evaluation, A/B testing, analytics

Architecture Proposal

High-level diagram

```
flowchart LR
  UI[Next.js Web App] -->|REST/JSON| API[Django + DRF]
  API --> DB[(Postgres)]
  API --> R[(Redis)]
```

```
API --> S3[(S3/MinIO)]
API -->|enqueue| W[Celery Workers]
W -->|LLM calls| MR[Model Router]
MR -->|local| L[Local LLM Server Ollama/vLLM]
MR -->|hosted| H[Hosted LLM Providers]
API --> OBS[Logging/Tracing]
```

Key design choices

- **Django as the "system of record"**: drafts, versions, audit, compliance decisions.
- **Next.js as the "creator cockpit"**: fast UX, future-proof product UI.
- **LLM boundary**: all model calls go through a **Model Router** service in Django (single choke point for costs, caching, safety, logging).

Data Model

Core entities

- `Workspace(id, name, plan, created_at)`
- `User(id, email, name, ...)` (Django auth)
- `Membership(user, workspace, role)`
- `BrandProfile(id, workspace, name, settings_json, facts_json, version, created_at)`
- `Brief(id, workspace, brand_profile, title, objective, audience, offer, constraints_json, created_at)`
- `Project(id, workspace, name, status, created_at)` *(this powers your management dashboard)*
- `Draft(id, project, brief, channel, content_json, rendered_text, status, version, created_at)`
- `DraftVersion(id, draft, content_json, rendered_text, version, created_at)`
- `ComplianceCheck(id, draft, overall_status, findings_json, confirmations_json, created_at)`
- `AuditEvent(id, workspace, actor, entity_type, entity_id, action, diff_json, created_at)`

Status flow (MVP)

```
IDEA -> BRIEFED -> GENERATED -> REVIEWED ->
COMPLIANCE_FLAGGED/COMPLIANT -> READY
```

API Sketch

Auth & tenancy

- `POST /api/workspaces/`
- `GET /api/me/`
- `POST /api/workspaces/{id}/members/` *(V1, optional)*

Brand

- `POST /api/brand-profiles/`
- `GET /api/brand-profiles/`
- `POST /api/brand-profiles/{id}/extract/` *(optional: generate tone/facts suggestions)*

Projects & briefs

- `POST /api/projects/`
- `PATCH /api/projects/{id}/` (status updates)
- `POST /api/briefs/`

Drafting

- `POST /api/drafts/generate/` body:
  - `project_id`, `brief_id`, `channel`, `mode=local|hosted`, `temperature`, etc.
- `GET /api/drafts/{id}/`
- `POST /api/drafts/{id}/revise/` (takes instructions + creates new DraftVersion)

Compliance

- `POST /api/drafts/{id}/compliance-check/`
- `POST /api/compliance-checks/{id}/confirm/` (confirmations/overrides)

Dashboard

- `GET /api/dashboard/summary` (counts + time-in-stage)
- `GET /api/projects/board` (projects grouped by status)

Milestones & Roadmap

Milestone 1 — Repo + infra (M)

- Docker compose: postgres, redis, minio, backend, frontend
- Env management, basic CI, formatting/linting

Milestone 2 — Domain foundation (M)

- Workspaces + Membership
- BrandProfile + Brief + Project + Draft models
- AuditEvent middleware

Milestone 3 — LLM generation vertical slice (L)

- Model Router (local inference first)
- Draft generation endpoints
- Next.js UI: create brief → generate IG/TikTok → view result

Milestone 4 — Compliance + versions (M)

- Compliance rules engine + UI findings
- Draft versioning + revise loop

Milestone 5 — Dashboard (S–M)

- Kanban-ish board + metrics summary

Implementation Tasks (step-by-step guidance format)

Step 0 — Prereqs

- Install: Docker Desktop, Python 3.11+, Node 20+, pnpm/npm
- Choose local LLM runner: **Ollama** (simplest)

Step 1 — Create repo structure

```
theluxe/
  backend/
  frontend/
  infra/
    docker-compose.yml
  docs/
```

Step 2 — Infra: docker-compose (MVP)

Services:

- `postgres`
- `redis`
- `minio`
- `backend` (Django)
- `frontend` (Next.js)

Step 3 — Django foundation

- `django-admin startproject config backend`
- Apps:
  - `tenancy`, `brand`, `content`, `compliance`, `audit`, `llm`
- DRF + JWT (or session auth for dev)

Step 4 — Model Router (local first)

- One interface: `generate(prompt, schema, mode)`
- Implement local call to Ollama HTTP API
- Add caching (hash(prompt+model+params) → response)

Step 5 — Draft generation

- Pydantic schemas for outputs:
  - `InstagramDraft{ hook, captions[], cta, hashtags[] }`
  - `TikTokDraft{ hook, script, shot_list[], on_screen_text[], caption, hashtags[] }`
- Prompt templates versioned in `backend/llm/prompts/`

Step 6 — Next.js cockpit

Pages:

- `/onboarding` (brand profile)
- `/projects` (board)
- `/projects/[id]` (brief + drafts)
- Draft viewer/editor with “revise” box

Step 7 — Compliance rules engine

- Start as deterministic rules (regex + heuristics + required confirmations)
- Output: `findings[]` with severity + rationale + suggested fixes

Starter Code / Scaffolding (key snippets)

1) Pydantic schema (backend/content/schemas.py)

```
from pydantic import BaseModel, Field
from typing import List

class InstagramDraft(BaseModel):
    hook: str
    captions: List[str] = Field(min_length=1, max_length=5)
    cta: str
    hashtags: List[str] = Field(default_factory=list)

class TikTokDraft(BaseModel):
    hook: str
    script: str
    shot_list: List[str] = Field(default_factory=list)
    on_screen_text: List[str] = Field(default_factory=list)
    caption: str
    hashtags: List[str] = Field(default_factory=list)
```

2) Model Router interface (backend/llm/router.py)

```
import hashlib, json, requests
from django.core.cache import cache

class ModelRouter:
    def __init__(self, ollama_url: str):
        self.ollama_url = ollama_url.rstrip("/")
```

```
    def _cache_key(self, model: str, payload: dict) -> str:
        raw = json.dumps({"m": model, "p": payload}, sort_keys=True).encode("utf-8")
        return "llm:" + hashlib.sha256(raw).hexdigest()

    def generate(self, model: str, prompt: str, temperature: float = 0.4) -> str:
        payload = {
            "model": model,
            "prompt": prompt,
            "stream": False,
            "options": {"temperature": temperature},
        }
        key = self._cache_key(model, payload)
        hit = cache.get(key)
        if hit:
            return hit

        r = requests.post(f"{self.ollama_url}/api/generate", json=payload, timeout=120)
        r.raise_for_status()
        text = r.json().get("response", "")
        cache.set(key, text, timeout=3600)
        return text
```

(We'll add robust retries/logging and schema-safe parsing in the next iteration.)

3) Generation endpoint (backend/content/views.py)

```
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
from .schemas import InstagramDraft, TikTokDraft
from llm.router import ModelRouter

router = ModelRouter(ollama_url="<http://ollama:11434>")

class GenerateDraft(APIView):
    def post(self, request):
        channel = request.data["channel"]
        brief = request.data["brief"]  # MVP: pass brief text; later: DB lookup
        model = request.data.get("model", "llama3.1:8b-instruct")

        if channel == "instagram":
            prompt = f"Return JSON for InstagramDraft. Brief:\\n{brief}"
            raw = router.generate(model=model, prompt=prompt)
```

```
        draft = InstagramDraft.model_validate_json(raw)
        return Response(draft.model_dump(), status=status.HTTP_200_OK)

    if channel == "tiktok":
        prompt = f"Return JSON for TikTokDraft. Brief:\\n{brief}"
        raw = router.generate(model=model, prompt=prompt)
        draft = TikTokDraft.model_validate_json(raw)
        return Response(draft.model_dump(), status=status.HTTP_200_OK)

    return Response({"error": "unsupported channel"}, status=400)
```

Risks & Tradeoffs

- **Next.js learning curve**: mitigated by keeping frontend thin initially and leaning on AI-assisted coding + strong API contracts.
- **Local model JSON reliability**: mitigate with:
  - strict schema prompting
  - “repair JSON” fallback
  - temperature low
- **Compliance expectations**: phrase it clearly as “risk flags + checklists”, not legal advice.

Next Actions (what you should do now)

1. Confirm the **frontend approach**:
   - Next.js **App Router** (recommended) + simple UI kit (shadcn/ui)
2. Tell me your dev OS (Windows/Linux) so I pick the best local LLM runner setup (Ollama is usually easiest).
3. Then I will guide you **step-by-step** starting with:
   - repo initialization
   - docker-compose
   - Django project + first migration
   - Next.js app + first page
   - first end-to-end “Generate IG Draft” flow

If you want to start immediately: reply with **“Start Step 1”** and your OS, and I’ll give you the exact commands and files to create.